Lab 9 Work Flow

Auth flow (login/register/refresh/reset/change)

- Login builds access + refresh tokens and persists refresh tokens (UserServiceImpl login + refreshToken; service/UserServiceImpl.java)
- Refresh token exchange at AuthController.java:

```java
@PostMapping("/logout")
public ResponseEntity<Map<String, String>> logout() {
    // In JWT, logout is handled client-side by removing token
    Map<String, String> response = new HashMap<>();
    response.put(key: "message", value: "Logged out successfully. Please remove token from client.");
    return ResponseEntity.ok(response);
}

@PutMapping("/change-password")
public ResponseEntity<Map<String, String>> changePassword(@Valid @RequestBody ChangePasswordDTO dto) {
    // 1. Get current user from SecurityContext
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
    String username = authentication.getName();

    // 2-5. Service handles verification, validation, hashing, and update
    userService.changePassword(username, dto);

    Map<String, String> response = new HashMap<>();
    response.put(key: "message", value: "Password changed successfully");
    return ResponseEntity.ok(response);
}
```

returning new access token

- Password change/forgot/reset handled in UserServiceImpl.java:

```java
        // Get user details
        User user = userRepository.findByUsername(loginRequest.getUsername())
                .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));

        return new LoginResponseDTO(
            token,
            refreshTokenValue,
            user.getUsername(),
            user.getEmail(),
            user.getRole().name()
        );
    }

    @Override
    public UserResponseDTO updateProfile(String username, UpdateProfileDTO dto) {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));

        // If email provided and changed, ensure it's unique (not used by another user)
        if (dto.getEmail() != null && !dto.getEmail().isBlank()) {
            String newEmail = dto.getEmail();
            if (!newEmail.equalsIgnoreCase(user.getEmail()) && userRepository.existsByEmail(newEmail)) {
                throw new DuplicateResourceException(message: "Email already exists");
            }
            user.setEmail(newEmail);
        }

        // Update full name if provided
        if (dto.getFullName() != null && !dto.getFullName().isBlank()) {
            user.setFullName(dto.getFullName());
        }

        User saved = userRepository.save(user);
        return convertToDTO(saved);
    }

    @Override
    public void deleteAccount(String username, String password) {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));

        if (!passwordEncoder.matches(password, user.getPassword())) {
            throw new IllegalArgumentException(s: "Password is incorrect");
        }

        user.setIsActive(isActive: false); // soft delete
        userRepository.save(user);
    }

    @Override
    public UserResponseDTO register(RegisterRequestDTO registerRequest) {
        // Check if username exists
        if (userRepository.existsByUsername(registerRequest.getUsername())) {
            throw new DuplicateResourceException(message: "Username already exists");
        }

        // Check if email exists
        if (userRepository.existsByEmail(registerRequest.getEmail())) {
            throw new DuplicateResourceException(message: "Email already exists");
        }

        // Create new user
        User user = new User();
        user.setUsername(registerRequest.getUsername());
        user.setEmail(registerRequest.getEmail());
        user.setPassword(passwordEncoder.encode(registerRequest.getPassword()));
        user.setFullName(registerRequest.getFullName());
        user.setRole(Role.USER);  // Default role
        user.setIsActive(isActive: true);

```

```
152
153         User savedUser = userRepository.save(user);
154
155         return convertToDTO(savedUser);
156     }
157
158     @Override
159     public UserResponseDTO getCurrentUser(String username) {
160         User user = userRepository.findByUsername(username)
161                 .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));
162
163         return convertToDTO(user);
164     }
165
166     @Override
167     public void changePassword(String username, ChangePasswordDTO dto) {
168         // 1. Get current user
169         User user = userRepository.findByUsername(username)
170                 .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));
171
172         // 2. Verify current password
173         if (!passwordEncoder.matches(dto.getCurrentPassword(), user.getPassword())) {
174             throw new IllegalArgumentException(s: "Current password is incorrect");
175         }
176
177         // 3. Check new password matches confirm
178         if (!dto.getNewPassword().equals(dto.getConfirmPassword())) {
179             throw new IllegalArgumentException(s: "New password and confirm password do not match");
180         }
181
182         // 4. Hash and update password
183         user.setPassword(passwordEncoder.encode(dto.getNewPassword()));
184         userRepository.save(user);
185     }
186
187     @Override
188     public ForgotPasswordResponseDTO forgotPassword(ForgotPasswordRequestDTO request) {
189         // Find user by email
190         User user = userRepository.findByEmail(request.getEmail())
191                 .orElseThrow(() -> new ResourceNotFoundException(message: "User with this email not found"));
192
193         // Generate reset token (UUID)
194         String resetToken = java.util.UUID.randomUUID().toString();
195
196         // Set token expiry (1 hour from now)
197         user.setResetToken(resetToken);
198         user.setResetTokenExpiry(java.time.LocalDateTime.now().plusHours(hours: 1));
199
200         userRepository.save(user);
201
202         // In production, you would send this token via email
203         // For now, we return it in the response
204         return new ForgotPasswordResponseDTO(
205             message: "Password reset token generated. In production, this would be sent to your email.",
206             resetToken
207         );
208     }
```

- Login response now carries refreshToken (dto/LoginResponseDTO.java)

User profile & account

- View/update profile and soft-delete account (auth required) in controller/UserController.java
- Service-side updates and soft delete in UserServiceImpl.java:

```java
    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private JwtTokenProvider tokenProvider;

    @Autowired
    private RefreshTokenRepository refreshTokenRepository;

    @Autowired
    private CustomUserDetailsService customUserDetailsService;

    @Override
    public LoginResponseDTO login(LoginRequestDTO loginRequest) {
        // Authenticate user
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                loginRequest.getUsername(),
                loginRequest.getPassword()
            )
        );

        SecurityContextHolder.getContext().setAuthentication(authentication);

        // Generate JWT access token
        String token = tokenProvider.generateToken(authentication);

        // Generate refresh token (7 days)
        String refreshTokenValue = java.util.UUID.randomUUID().toString();
        RefreshToken rt = new RefreshToken();
        rt.setUser(userRepository.findByUsername(loginRequest.getUsername())
                .orElseThrow(() -> new ResourceNotFoundException(message: "User not found")));
        rt.setToken(refreshTokenValue);
        rt.setExpiryDate(LocalDateTime.now().plusDays(days: 7));
        refreshTokenRepository.save(rt);

        // Get user details
        User user = userRepository.findByUsername(loginRequest.getUsername())
                .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));

        return new LoginResponseDTO(
            token,
            refreshTokenValue,
            user.getUsername(),
            user.getEmail(),
            user.getRole().name()
        );
    }

    @Override
    public UserResponseDTO updateProfile(String username, UpdateProfileDTO dto) {
        User user = userRepository.findByUsername(username)
                .orElseThrow(() -> new ResourceNotFoundException(message: "User not found"));

        // If email provided and changed, ensure it's unique (not used by another user)
        if (dto.getEmail() != null && !dto.getEmail().isBlank()) {
            String newEmail = dto.getEmail();
            if (!newEmail.equalsIgnoreCase(user.getEmail()) && userRepository.existsByEmail(newEmail)) {
                throw new DuplicateResourceException(message: "Email already exists");
            }
            user.setEmail(newEmail);
        }

        // Update full name if provided
        if (dto.getFullName() != null && !dto.getFullName().isBlank()) {
            user.setFullName(dto.getFullName());
        }

        User saved = userRepository.save(user);
        return convertToDTO(saved);
    }
```

Admin operations

- List users, update role, toggle active status in controller/AdminController.java Implemented in UserServiceImpl.java:

```java
@Override
public ForgotPasswordResponseDTO forgotPassword(ForgotPasswordRequestDTO request) {
    // Find user by email
    User user = userRepository.findByEmail(request.getEmail())
            .orElseThrow(() -> new ResourceNotFoundException(message: "User with this email not found"));

    // Generate reset token (UUID)
    String resetToken = java.util.UUID.randomUUID().toString();

    // Set token expiry (1 hour from now)
    user.setResetToken(resetToken);
    user.setResetTokenExpiry(java.time.LocalDateTime.now().plusHours(hours: 1));

    userRepository.save(user);

    // In production, you would send this token via email
    // For now, we return it in the response
    return new ForgotPasswordResponseDTO(
        message: "Password reset token generated. In production, this would be sent to your email.",
        resetToken
    );
}

@Override
public void resetPassword(ResetPasswordRequestDTO request) {
    // Find user by reset token
    User user = userRepository.findByResetToken(request.getResetToken())
            .orElseThrow(() -> new ResourceNotFoundException(message: "Invalid reset token"));

    // Check if token is expired
    if (user.getResetTokenExpiry() == null ||
        user.getResetTokenExpiry().isBefore(java.time.LocalDateTime.now())) {
        throw new IllegalArgumentException(s: "Reset token has expired");
    }

    // Check new password matches confirm
    if (!request.getNewPassword().equals(request.getConfirmPassword())) {
        throw new IllegalArgumentException(s: "New password and confirm password do not match");
    }

    // Update password
    user.setPassword(passwordEncoder.encode(request.getNewPassword()));

    // Clear reset token
    user.setResetToken(resetToken: null);
    user.setResetTokenExpiry(resetTokenExpiry: null);

    userRepository.save(user);
}
```

Customers

- CRUD guarded by roles via controller/CustomerRestController.java with rules set in security/SecurityConfig.java

Refresh token storage

- Entity & repo: entity/RefreshToken.java, repository/RefreshTokenRepository.java

DTOs added

- UpdateProfileDTO, UpdateRoleDTO, RefreshTokenDTO (dto folder)

Endpoints ready for testing

- Auth: /api/auth/login, /api/auth/register, /api/auth/refresh, /api/auth/me, /api/auth/change-password, /api/auth/forgot-password, /api/auth/reset-password, /api/auth/logout
- Users: /api/users/profile (GET/PUT), /api/users/account (DELETE)
- Admin: /api/admin/users (GET), /api/admin/users/{id}/role (PUT), /api/admin/users/{id}/status (PATCH)
- Customers: /api/customers (GET, POST, PUT, DELETE), /api/customers/{id} (GET)